

SQL - Week 2

Load your Excel file and create a searchable SQLite table

DataFrame Setup

'orders': name of newly created table inside our SQL database.

conn: The active connection link pointing to our SQLite database (analysis.db).

```
In [47]: import pandas as pd
import sqlite3

df = pd.read_excel('SQL_Sales_Dataset_200_Rows.xlsx')
conn = sqlite3.connect('analysis.db')
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_') # This turns
df.to_sql('orders', conn, index=False, if_exists='replace')
```

Out[47]: 200

1) Total Revenue by Category

Calculate the gross revenue and order count for each product category:

Step 1: Select and From

```
In [48]: query = """
          SELECT category, total_price      -- these are columns name to select
          FROM orders                      -- from our sql 'orders'
          """
display(pd.read_sql(query, conn).head())    # .head() shows just the first 5
```

	category	total_price
0	Clothing	2394
1	Clothing	13581
2	Clothing	5230
3	Furniture	2920
4	Furniture	1096

Step 2: Adding (GROUP BY)

To combine multiple rows into single categories instead of listing every individual order, use GROUP BY.

```
In [49]: query = """
        SELECT category
        FROM orders

        GROUP BY category
        """
        display(pd.read_sql(query, conn))
```

	category
0	Clothing
1	Electronics
2	Furniture
3	Grocery

Step 3: Adding Aggregations (COUNT and SUM)

Adding math functions inside the select statement to calculate metrics for each of those grouped categories.

```
In [50]: query = """
        SELECT
            category,
            COUNT(order_id) AS total_orders,
            SUM(total_price) AS total_revenue
        FROM orders

        GROUP BY category
        """
        display(pd.read_sql(query, conn))
```

	category	total_orders	total_revenue
0	Clothing	40	484259
1	Electronics	52	549302
2	Furniture	59	714399
3	Grocery	49	672147

Step 4: Sorting the Output (ORDER BY DESC)

Adding ORDER BY to sort our summarized results so the highest revenue categories appear at the top.

```
In [51]: query = """
```

```

SELECT
    category,
    COUNT(order_id) AS total_orders,
    SUM(total_price) AS total_revenue
FROM orders

GROUP BY category

ORDER BY total_revenue DESC      -- DESC means in descending order, ORDER BY m
"""
display(pd.read_sql(query, conn))

```

	category	total_orders	total_revenue
0	Furniture	59	714399
1	Grocery	49	672147
2	Electronics	52	549302
3	Clothing	40	484259

SQL Core Clauses and Aggregations Breakdown

SELECT: Specifies the exact columns you want to extract from your database table.

WHERE: Filters rows based on specific conditions before any grouping or calculations take place.

GROUP BY: Combines rows that share the same value in specified columns into summary groups.

ORDER BY: Sorts your final result set in ascending (ASC) or descending (DESC) order.

COUNT(): An aggregate function that returns the total count of rows or non-null values.

SUM(): An aggregate function that calculates the total sum of a numeric column.

AVG(): An aggregate function that computes the mathematical average (mean) of a numeric column.

```

In [52]: # Combined query utilizing SELECT, WHERE, GROUP BY, ORDER BY, and aggregations

query = """
    SELECT
        category,                                -- Columns to display
        COUNT(order_id) AS total_orders,         -- Count aggregation
        SUM(total_price) AS gross_revenue,       -- Sum aggregation
        AVG("Unit Price") AS avg_unit_price     -- Double quotes around "Unit Price"
    FROM orders                                  -- Source table

    WHERE region = 'East'                       -- Filtering rows first

```

```

GROUP BY category          -- Grouping by product category

ORDER BY gross_revenue DESC -- Sorting results from highest to lowest rev
"""

# Execute and display the results
display(pd.read_sql(query, conn))

```

	category	total_orders	gross_revenue	avg_unit_price
0	Grocery	16	226818	0.0
1	Furniture	15	152365	0.0
2	Clothing	10	146363	0.0
3	Electronics	10	93704	0.0

2) Joins, subqueries, CASE statements • Assignments:

```

In [58]: import pandas as pd
import sqlite3

# 1. Connect to your SQLite database
conn = sqlite3.connect('analysis.db')

# 2. Load both tables from your Excel files into SQLite
# Loading the customers and orders from the second workbook
df_customers = pd.read_excel('Other sheet.xlsx', sheet_name='Customers')
df_orders = pd.read_excel('Other sheet.xlsx', sheet_name='Orders')

df_customers.to_sql('customers', conn, index=False, if_exists='replace')
df_orders.to_sql('orders2', conn, index=False, if_exists='replace')

```

Out[58]: 50

1. Joins (Combining Tables) Joins connect two tables together using a common column (in this case, customer_id).

Same Above - using 'o' & 'c'

```

In [67]: # Inner Join: Returns matching rows from both tables
query_inner = """
SELECT
    o.order_id,          -- Selects order ID from the orders table (al
    c.customer_name,     -- Selects customer name from the customers t
    c.segment,           -- Selects customer segment from the customer
    o.sales,             -- Selects sales amount from the orders table
    o.region             -- Selects region from the orders table
FROM orders2 AS o        -- Specifies the main table ('orders2') and n

```

```

JOIN customers AS c          -- Brings in the 'customers' table and nickna
ON o.customer_id = c.customer_id -- The common link connecting both tables toge
LIMIT 5                      -- Restricts the output to only the first 5 r
"""

display(pd.read_sql(query_inner, conn))

```

	order_id	customer_name	segment	sales	region
0	ORD_0001	Alice Smith	Home Office	421.30	South
1	ORD_0002	Laura Croft	Home Office	108.63	East
2	ORD_0003	Laura Croft	Home Office	1080.46	Central
3	ORD_0004	Quentin Tarantino	Corporate	210.79	East
4	ORD_0005	Julia Roberts	Home Office	687.04	West

```

In [69]: # Left Join: Returns all customers, even those who haven't placed any orders
query_left = """

SELECT
    c.customer_id,
    c.customer_name,
    o.order_id,
    o.sales
FROM customers AS c          -- base

LEFT JOIN orders2 AS o      -- joining order2 on base on left

ON c.customer_id = o.customer_id -- criteria

LIMIT 5
"""

display(pd.read_sql(query_left, conn))

```

	customer_id	customer_name	order_id	sales
0	CUST_001	Alice Smith	ORD_0001	421.30
1	CUST_002	Bob Jones	ORD_0020	478.67
2	CUST_002	Bob Jones	ORD_0033	561.22
3	CUST_002	Bob Jones	ORD_0039	440.65
4	CUST_003	Charlie Brown	ORD_0010	1058.50

Example Comparison:

An Inner Join drops customers who haven't placed any orders.

A Left Join keeps those non-ordering customers visible, letting you easily spot who hasn't bought anything yet.

If a customer in the customers table never placed an order, the LEFT JOIN still shows their name, but puts None (NULL) in the columns for order_id and sales because no match exists in the orders table.

3) Subqueries (Nested Queries)

A subquery performs an inner calculation first, and the outer query uses that result.

```
In [87]: # Find all orders where sales are higher than the overall average sales
# Re-open the database connection
conn = sqlite3.connect('analysis.db')
query_sub = """

    SELECT order_id, customer_name, total_price
    FROM orders

    WHERE total_price > (
        SELECT AVG(total_price)
        FROM orders
    )

    ORDER BY total_price DESC
    LIMIT 5
"""
display(pd.read_sql(query_sub, conn))
```

	order_id	customer_name	total_price
0	1125	Lynn Garrison	47940
1	1019	Debbie Turner	44990
2	1168	Megan Charles	44620
3	1160	Michelle Beltran	42471
4	1120	Rick Sanford	41211

4) CASE Statements (Conditional Logic)

A CASE statement acts as an IF-THEN rule inside your SQL query to categorize data on the fly.

```
In [88]: # Categorize orders into value tiers based on the sales amount
# Re-open the database connection
conn = sqlite3.connect('analysis.db')
```

```

query_case = """
SELECT
    order_id,
    total_price,
    CASE
        WHEN total_price > 1400 THEN 'High Value'
        WHEN total_price BETWEEN 1348 AND 1400 THEN 'Medium Value'
        ELSE 'Standard Value'
    END AS total_price_category
FROM orders

ORDER BY total_price DESC
LIMIT 5
"""
display(pd.read_sql(query_case, conn))

```

	order_id	total_price	total_price_category
0	1125	47940	High Value
1	1019	44990	High Value
2	1168	44620	High Value
3	1160	42471	High Value
4	1120	41211	High Value

END: Closes the conditional CASE statement. Just like parentheses

AS sales_category: Renames the resulting temporary column to sales_category in your final query output.

6) Query a sample database to find top customers, average order values.

```

In [89]: import pandas as pd
import sqlite3

# 1. Connect to your SQLite database
conn = sqlite3.connect('analysis.db')

# 2. Write the query combining a JOIN with aggregate functions (SUM, AVG, COUNT)
query = """
SELECT
    o.customer_name,                -- Name of the customer from the customer
    COUNT(o.quantity) AS total_orders, -- Total number of orders they placed
    SUM(o.total_price) AS total_price,   -- Total revenue generated from th
    AVG(o.total_price) AS avg_order_value -- Average order value (AOV) per t
FROM orders AS o                    -- base

-- JOIN customers AS c ON o.customer_id = c.customer_id -- joining customers

```

```
GROUP BY o.customer_name          -- for getting top customers

ORDER BY total_price DESC         -- for getting top customers

LIMIT 5                          -- Restricts the output to the top 5 cust
"""

# 3. Execute query and display results
display(pd.read_sql(query, conn))

# Close connection when finished
conn.close()
```

	customer_name	total_orders	total_price	avg_order_value
0	Lynn Garrison	1	47940	47940.0
1	Debbie Turner	1	44990	44990.0
2	Megan Charles	1	44620	44620.0
3	Michelle Beltran	1	42471	42471.0
4	Rick Sanford	1	41211	41211.0

ProgrammingError: Cannot operate on a closed database.

This happens because you closed the database connection (`conn.close()`) in a previous cell, but tried to run another query afterward using that same `conn` variable.

To fix it, simply re-run your connection setup cell to open a fresh link to the database: